

Stretch: Launching the Audio App to 5,000 Gig Workers in One Weekend

The current build (Task 3) is a local Flask dev server with SQLite storage and audio files saved to disk — good enough to prove the concept end-to-end, but several parts of it would break well before reaching 5,000 submissions. Below is what breaks first, in rough order, and what I'd change before a real launch.

What breaks first

1. The dev server itself. `app.run(debug=True)` is Flask's built-in single-threaded development server — it explicitly warns "do not use in production." At any real concurrency (more than a handful of simultaneous submitters), requests start queueing and timing out. This is the very first thing that falls over, likely within the first few minutes of traffic.

2. SQLite write locking. SQLite allows only one writer at a time; concurrent submissions from multiple gig workers would queue up or throw `database is locked` errors. With 5,000 workers submitting over a weekend, even modest overlap (a few people submitting in the same second) would start failing silently or with 500 errors.

3. Local disk storage for audio files. Audio files currently save to `audio_app/uploads/` on the server's own disk. This has two problems at scale: (a) a single server's disk will fill up — even at a conservative 500KB average per recording, 5,000 submissions is ~2.5GB, manageable alone, but if this is running as a single instance with no autoscaling, there's no redundancy or backup, and a disk failure loses everything; (b) if the app needs multiple server instances to handle load, each instance only sees files uploaded to itself, so "view all submissions" would only show a fraction of the data depending on which instance served the "list" request.

4. No duplicate detection. Right now, nothing stops the same worker from submitting 5 times, accidentally or intentionally (e.g. testing the mic, or trying to get

counted multiple times for an incentive). At 5,000 workers this could meaningfully inflate the numbers with no way to tell real submissions from repeats.

5. No upload validation or size limits beyond the 25MB cap. A malicious or confused user could upload something that isn't audio at all (renamed file), or a very long recording that eats disk space and takes a long time to process through librosa/ffmpeg, blocking other requests on the single-threaded dev server.

6. No monitoring or error visibility. If ffmpeg conversion fails for some worker's device/browser combination (e.g. an unusual codec), the current code returns a JSON error to the browser, but nobody on the ConsultBae side would know unless they're actively watching logs. At 5,000 submissions, silent failures would go unnoticed until someone complained.

What I'd change before launch

- **Swap SQLite for Postgres** (e.g. managed via Render/Railway/RDS) to handle concurrent writes safely, and run the Flask app behind a real WSGI server (gunicorn) with multiple workers, ideally behind a load balancer.
- **Move audio storage to object storage** (S3, Cloudflare R2, or similar) instead of local disk — this solves both the durability problem and the multi-instance visibility problem, since every instance reads/writes to the same bucket.
- **Add duplicate detection** on (name, phone) with a soft warning ("You've already submitted — are you sure you want to submit again?") rather than a hard block, since legitimate retries (bad first recording) should still be allowed.
- **Move audio analysis (librosa/ffmpeg) off the request path** into a background job queue (e.g. Celery + Redis, or a simple task queue) so a slow/large file doesn't block other people's submissions. The user gets an immediate "received, processing" response instead of waiting on the analysis synchronously.
- **Add basic rate limiting per phone number/IP** to blunt accidental multi-submission storms and reduce abuse risk.
- **Add structured logging + a simple alert** (e.g. Slack webhook via the same n8n instance from Task 2) for any submission that fails analysis, so failures are visible in near-real-time instead of buried in server logs.
- **Cost consideration:** object storage + a small managed Postgres + a couple of small compute instances for a single weekend of 5,000 submissions is a low, predictable cost (likely under \$20-30 for the weekend on any major cloud/PaaS)

— cheap insurance against the app falling over during the actual launch window.